Constrained policy recovery for the Pendubot via Curriculum Learning and Logarithmic Barrier Penalties

Davide Molinaroli

Abstract

This project, which was proposed for a possible participation at the 3rd AI Olympics with RealAIGym at ICRA 2025, explores the recovery of a simulated Pendubot system after it falls from its unstable upright equilibrium. The goal was to design a strategy capable of bringing the system back to the balanced state while staying within position and velocity constraints. The attempts focused on modifying the simulation environment to disturb the system, shaping the reward function based on constraint violation and applying curriculum learning to first learn the swing-up maneuver and then discover how to re-stabilize after falling. The outcome is a policy that is able to recover from a fall within the joint position constraints enforced by the competition.

1. Introduction

In the context of reinforcement learning for robotic systems, the sim2real gap and constraint satisfaction are critical points of discussion. This work was developed to reason about such issues and with the objective of contributing to the EvolSac team submission (Calì et al., 2024) for the third AI Olympics With RealAIGym.

Indeed, in last year's competition, on the Pendubot system, stabilization unexpectedly failed due to sim2real discrepancies. Although the policy successfully managed to re-stabilize the system, the recovery trajectory violated state constraints.

Consequently, this project adresses the problem in three ways:

1. Modifying the training environment of the existing Soft Actor-Critic (SAC) agent to include the fall of the system by applying a torque disturbance for one step and then turning off the controller for one second;
2. Tuning a physics-inspired reward function to punish constraint violation by introducing penalties that resemble logarithmic barrier functions;
3. First training the agent in a default environment to learn the swing-up maneuver, then resuming the training procedure in a different environment which incorporates the disturbance and the modified reward function. This is a very simple case of Curriculum Learning (Narvekar et al., 2020): the agent is trained on environments of increasing difficulty.

This paper is organized as follows: Section 2 formalizes the system, goal and initial setup; Section 3 discusses about the design of the disturbance; Section 4 about the modified reward function; Section 5 about the results and Section 6 about conclusions and possible improvements.

2. Problem definition

2.1. System

The analyzed system is a 2-dof underactuated pendulum. It consists of two masses, m_1 and m_2, each one attached to rods of length l_1 and l_2. A revolute joint connects the first rod to a ground frame and another revolute pair couples the second rod to the first one. Fig. 1 shows a schematic representation of the system. The overall motion is planar. In the pendubot configuration, only the first revolute joint is actuated through a torque-controlled motor, while the second one is passive and is only used as an encoder. For the control task, the state space representation $[\theta_1(t)\ \theta_2(t)\ \dot{\theta}_1(t)\ \dot{\theta}_2(t)]$ was used as state x_t for the agent. The torque limit for the motor is set to $\tau_l = 10Nm$.

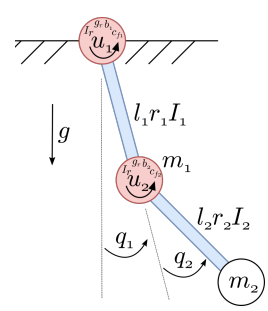

Figure 1. Double pendulum schematic. In the pendubot configuration, u_1 is the only actuated motor and the second joint is passive.

2.2. Goal

In the scope of this project, the behavior to implement is as follows:

1. Perform the swing-up maneuver, namely stabilize the system around $x_d = [\pi\ 0\ 0\ 0]$;
2. Simulate the fall of the system;
3. Stabilize again around x_d while satisfying the constraints $\theta_i \in [-4\pi, 4\pi],\ \dot{\theta}_i \in [-20, 20],\ i = 1, 2$. For the sake of simplicity, we consider only constraints on the joint angles, neglecting velocity restrictions. In fact, the obtained policies induce trajectories in which velocities remain within those bounds.

2.3. Initial setup

In the initial setup, the SAC agent is trained in the default environment with no disturbances and using a physics-inspired reward function defined as

$$R(x_t) = \begin{cases} V(x_t) & \text{if } y < 0.7 \cdot l_{tot} \\ V(x_t) + 2(1 + \cos(\theta_2))^2 - 2T(x_t) & \text{if } y \geq 0.7 \cdot l_{tot} \end{cases}$$

where V and T are respectively the potential and kinetic energy of the system, y is the end effector position and l_{tot} is the sum of the lengths of the two links. The control line is defined as $0.7 \cdot l_{tot}$. Above the control line, the agent is rewarded not only by the potential energy of the system, but also by keeping the second link aligned with the first one. Moreover, it is penalized by its kinetic energy. This energy-based reward function provides a smooth and interpretable signal to the agent, which learns to swing up by maximizing its potential energy withouth oscillations thanks to the kinetic penalty. A noisy reset function is used to initialize the pendubot in random initial conditions around the downward equilibrium to further encourage exploration, which SAC already includes. To boost training speed, episodes terminate whenever the pendubot falls below the control line. The agent is trained in 25 parallel environments for a total of $2.5 \cdot 10^6$ steps. The learning rate is set to 0.005, while other SAC parameters have been kept as the default ones. A time step of $0.01s$ is used and the episode length is set to 1500 steps. Every 5000 steps the agent is evaluated in a single non-stochastic environment with initial condition $x_0 = [0, 0, 0, 0]$ and deterministic policy output. The agent that achieves the best return in this environment is selected as the optimal one for the swing up maneuver.

Fig. 2, 3 and 4 show the performance and average returns of the models obtained from evaluation and when training ends, respectively. As expected, the former achieves better results, but the latter proved to be more robust the disturbance designed in Sec. 3 and provided more stable training in the second part of the curriculum. Such model is referred to as *model1*.

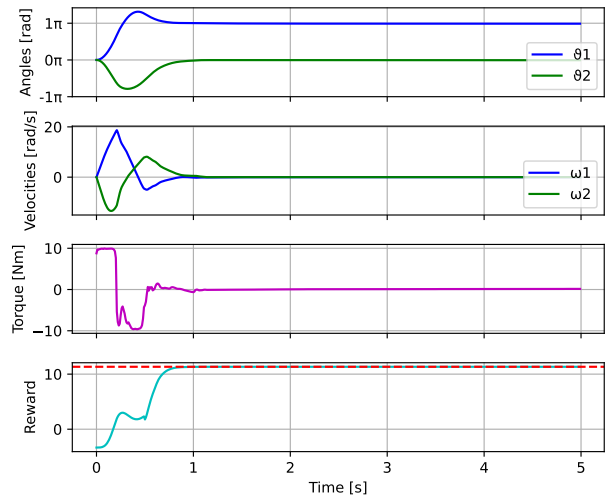

Figure 2. State evolution of the best evaluation model.

3. Disturbance design

Three approaches to disturbance design have been attempted:

1. turning off the controller for one second;
2. applying a constant torque disturbance for 5 steps;
3. applying a constant torque disturbance τ_d for 1 step, then turning off the controller for one second.

Approaches 2 and 3 apply the torque by overwriting the agent's actions. It became immediately clear that activating the disturbance after a certain state dependent condition is met was not the way to go. For example, applying the disturbance once the pendubot rised above the control line, resulted in the agent learning to keep the end effector just below that line. So, it was decided to disturb the system after 1 second within the episode. This way, the agent is forced to actually swing up before falling, which is the target behaviour. Resuming training with approach 1 was discarded because it resulted in a perfect stabilization around x_d, so that the gravity force would not induce any torque onto the two joints. Approach 2 was discarded as it simply resulted in the agent learning to cancel the effect of the disturbance right after the 5 steps, thus not simulating the actual fall. Approach 3 simulates more accurately the fall of the system that was shown to me by Niccolò. Three values of τ_d have been considered: $\tau_{d1} = 0.5 \cdot \tau_l$, $\tau_{d2} = 0.75 \cdot \tau_l$, $\tau_{d3} = \tau_l$. *model1* turned out to be already robust to τ_{d1} and it pretty much did not violate the state constraints, as shown in Fig. 5. The response to τ_{d3} shows that τ_{d3} makes θ_1 evolve outside its constraint while the controller is disabled

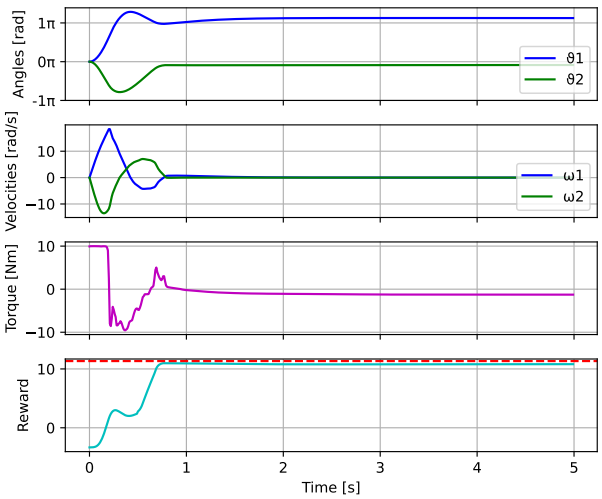

Figure 3. State evolution of *model1*, namely the one obtained after $2.5 \cdot 10^6$ training steps.

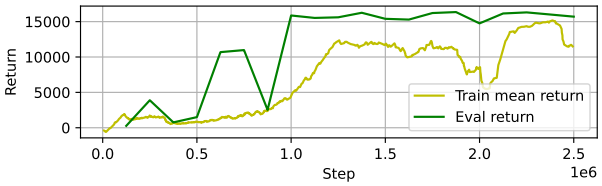

Figure 4. Training and evaluation learning curves in the initial setup.

(between seconds 1 and 2), thus making it impossible to not violate said constraint. Moreover, the real pendubot violated the bound on θ_2, so this is not the exact behaviour that we are looking for. The remaining value τ_{d2} accurately simulates the desired disturbace: the pendubot falls and then swings the second link more than two times. It is worth noting that it cannot stabilize after falling.

So, approach 3 with τ_{d2} is the selected disturbance, which is applied after 1 second within an episode and overwrites the agent's actions for 1 second. It is interesting to notice that resuming training in a new environment that incorporates this disturbance and without modifying the reward function doesn't lead to the recovery in $2.5 \cdot 10^6$ steps. This suggests that the recovery is most likely to happen if the agent is guided by a different reward function, such as one that penalizes the violation of the constraints. It would have been interesting to first obtain a recovery maneuver that violates constraints without modifying the reward function and then add a third phase in the curriculum where the penalty terms described in Sec.4 are used. Additionally, resetting the pendubot in more aggressive initial conditions during training was tried but did not yeild signs of recovery from the disturbance.

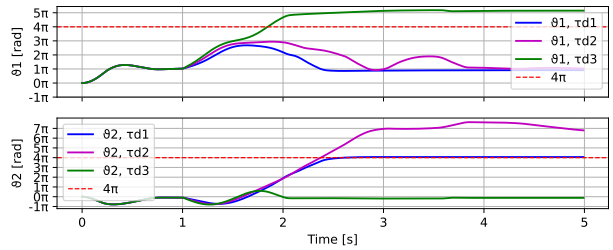

Figure 5. Response of *model1* to τ_{d1}, τ_{d2} and τ_{d3}.

4. Reward function design

The penalty terms introduced in the reward function take inspiration from logarithmic barrier approaches such as the Interior Point Method (Kim et al., 2007) and Control Barrier Functions (Misyats et al., 2025). The core idea is that a properly parametrized logarithmic penalty term should: (1) not decrease too much the reward if the state is close to and inside the constraint boundary, thus allowing high performance and (2) significantly decrease the reward once the constraint is violated. Moreover, logarithmic functions are smooth, which should intuitively encourage continuity in the reward distribution. So, the penalty takes the form

$$p(\theta) = 2 \cdot \log\left(-|\theta| + 4\pi \ + \ \theta_{slack}\right)$$

The reward function becomes

$$R'(x_t) = \begin{cases} R(x_t) + p(\theta_1) + p(\theta_2) & \text{if } y < 0.7 \cdot l_{tot} \\ R(x_t) + p(\theta_1) + k \cdot p(\theta_2) + 2cos(\theta_2) & \text{if } y \geq 0.7 \cdot l_{tot} \end{cases}$$

In particular, θ_{slack} and k are used to tune how strict the penalty is with respect to the constraint. The term $2cos(\theta_2)$ in $R'(x_t)$ is added when the pendubot is above the control line and it encourages the correct alignment of the second link even if the constraint is being violated. Indeed, since the bound $\pm 4\pi$ for θ_2 represents good stabilization when $\theta_1 = \pm\pi$, the policy should be able to explore around $\theta_2 = \pm 4\pi$ without being punished too much. The same idea applies to θ_{slack} and k. The former, when positive, delays the bigger punishments to higher values of θ. The latter, when smaller than 1, weakens the penalty when the system is above the control line. Again, the idea is the following: since stabilization may occur near the constraint boundary, penalizing this region too strongly could lead to suboptimal

policies, where the agent prioritizes staying well inside the limits over fast recovery. The necessity to tune the penalty function is the key limitation of this approach, where constraints are treated as "soft constraints". Formal frameworks, such as Interior-Point Policy Optimization (IPO) (Liu et al., 2019), directly incorporate the penalty in the policy's loss function, following the traditional Lagrangian approach for constrained optimization.

Three scenarios for $R'(x_t)$ are considered:

$$\begin{aligned} \textit{Loose:} \quad & \theta_{1,\text{slack}}{=}0, \quad \theta_{2,\text{slack}} = \tfrac{\pi}{2}, \quad k{=}0.5 \\ \textit{Strict:} \quad & \theta_{1,\text{slack}}{=}0, \quad \theta_{2,\text{slack}} = 0, \quad k{=}1 \\ \textit{Compromise:} \quad & \theta_{1,\text{slack}}{=}0, \quad \theta_{2,\text{slack}} = \tfrac{\pi}{6}, \quad k{=}1 \end{aligned}$$

and their associated penalties are shown in Fig. 6.

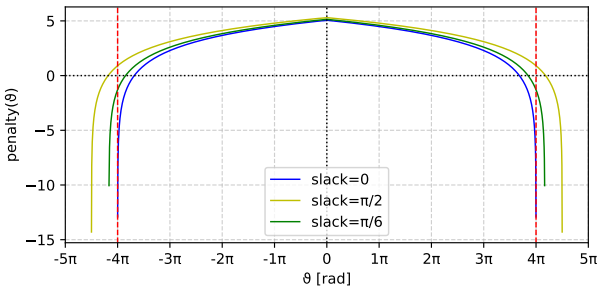

Figure 6. Logarithmic penalty functions.

5. Results

As already mentioned, *model1* is used as the initial SAC agent to train in the new environment, which incorporates the disturbance described in Sec.3 and the reward functions of Sec.4. The training parameters remain the same, but now evaluation is carried out every 1500 steps. Indeed, due to the disturbance and the stochastic nature of SAC, training appeared to be less stable than the first phase, so more evaluations were necessary to capture the best policy. Moreover, the reset function for the *Compromise* scenario was set to the zero reset function, which starts every episode at $x_0 = [0, 0, 0, 0]$, for more stability. Indeed, here the agent already knows how to swing-up, so it is not really necessary to inject more exploration with random initialization.

5.1. *Loose* scenario

In the *Loose* scenario, the policy receives little penalization if θ_2 gets close to 4π when above the control line. Indeed, the best evaluation model, shown in Fig. 7, recovers by rotating against the direction of the disturbance, making the second link swing two times around the first link. The state evolution slightly violates the constraint on θ_2. On the other hand, the final model after training, shown in Fig. 8, recovers by taking advantage of the speed gained after the disturbance and keeps rotating θ_1 in that direction, until it reaches 3π and then applies a negative torque to align the two links. This behaviour is suboptimal for this reward function, hence the lower return. It is suboptimal because this setup encourages to swing θ_2 near the boundary when above the control line and penalizes higher values of θ_1. Nonetheless, it allows to discover a faster way to recover. Indeed, this model fully recovers at $t = 2.70s$ against $t = 2.81s$ of the evaluation one. Notice, however, how the trained model is not able to exactly stabilize, as it oscillates very slowly around x_d. Fig. 9 shows the learning curves for this scenario.

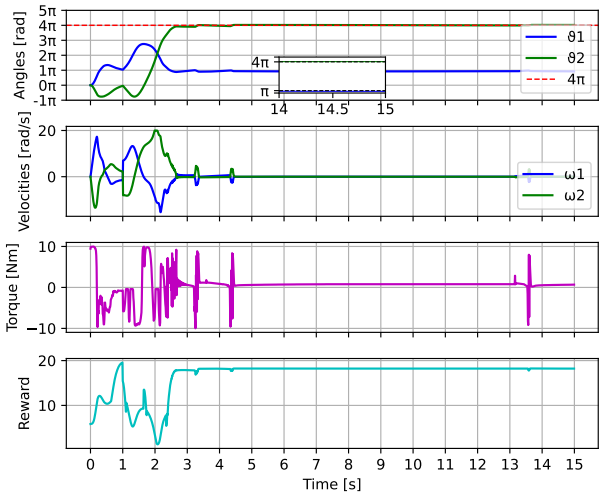

Figure 7. Best evaluation model, *Loose* scenario.

5.2. *Strict* scenario

In the *Strict* scenario, the policy is highly penalized near the boundaries of the constraints and the penalty is not reduced when above the control line. This forces the agent to learn the recovery by swinging the first link until $\theta_1 = 3\pi$ while bringing θ_2 to 2π. The best evaluation model, shown if Fig. 10, exhibits an unusual behaviour: after applying the negative torque to bring θ_1 to 3π, it applies impulses of $\pm\tau_l$ to finalize the stabilization, making the system look jittery and without precisely reaching x_d. The final trained model, which is shown in Fig. 11, applies a less intense positive torque signal to precisely stabilize in x_d. Thus, the result is slightly slower but more accurate. More precisely, the evaluation model fully recovers at $t = 2.80s$ and the trained one at $t = 2.97s$. The return appears to be lower in the trained model because of the speeds it achieves above

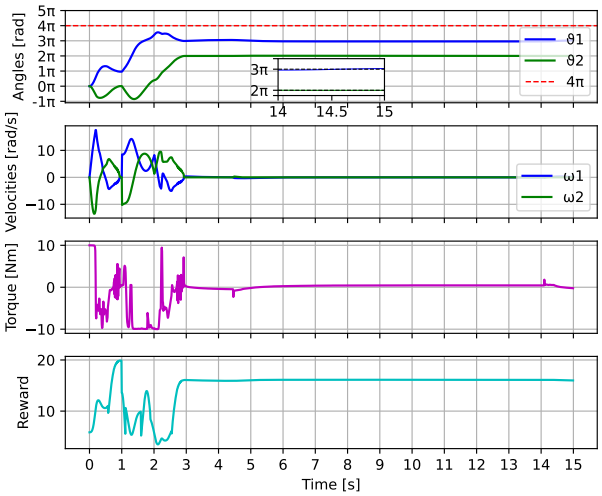

Figure 8. Trained model, *Loose* scenario.

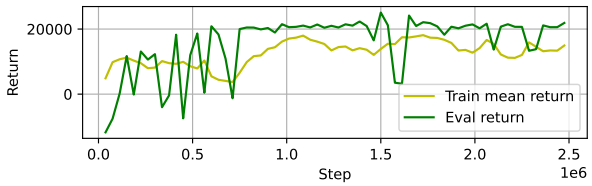

Figure 9. Learning curves, *Loose* scenario.

the control line, thus it receives a bigger kinetic penalty than the evaluation model. The learning curves, shown in Fig. 12, show that the initial returns are very low. This is expected because, as seen in Fig. 5, *model1* cannot stabilize from τ_{d2} and it violates the state constraints, leading to high penalties. This might make the training process unstable, thus producing the jittery and slower behaviours.

5.3. *Compromise* scenario

The final scenario tries to address the unusual behaviour seen in the *Strict* one by introducing $\theta_{2,slack} = \pi/6$. This makes the penalty more forgiving to the violation of the constraint on θ_2 and, together with the usage of the zero reset function, is an attempt to stabilize the training process. Indeed, Fig. 14 shows the learning curves, which appear to be somewhat more stable compared to Fig. 12 as the train and evaluation returns acually increase in the first $2 \cdot 10^6$ steps, with the exception of some evaluation episodes. Once again, the initial negative returns prove that the agent learns to not violate the constraints. Fig. 13 shows the performance of the best evaluation model, which achieves full recovery at $t = 2.61s$, making it the fastest agent. The accuracy is not perfect, as it stabilizes at $\theta_1 = 188°$ and $\theta_2 = 351°$. The initial stabilization is also not perfect, but this is ok considering that in the real world scenario it is unknown why and when the pendubot can fall.

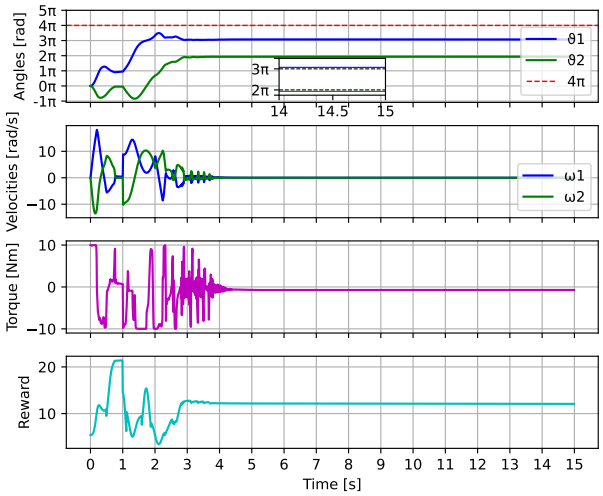

Figure 10. Best evaluation model, *Strict* scenario.

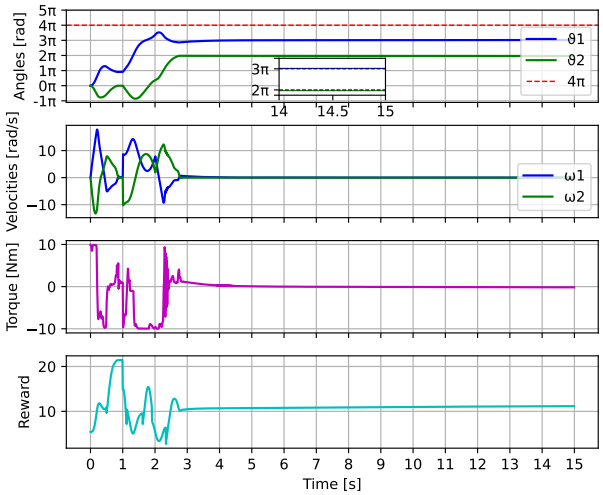

Figure 11. Trained model, *Strict* scenario.

5.4. Robustness

The robustness of the best evaluation agents for each scenario is shortly assessed. The assesment is performed by applying the disturbance and deactivation in time instants t_d different than 1 second and also by changing the sign of τ_{d2}. Fig. 15 shows that when $\tau_{d2} > 0$ then the agents can recover

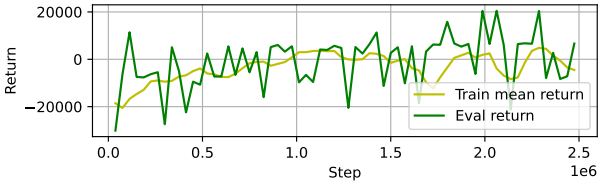

Figure 12. Learning curves, *Strict* scenario.

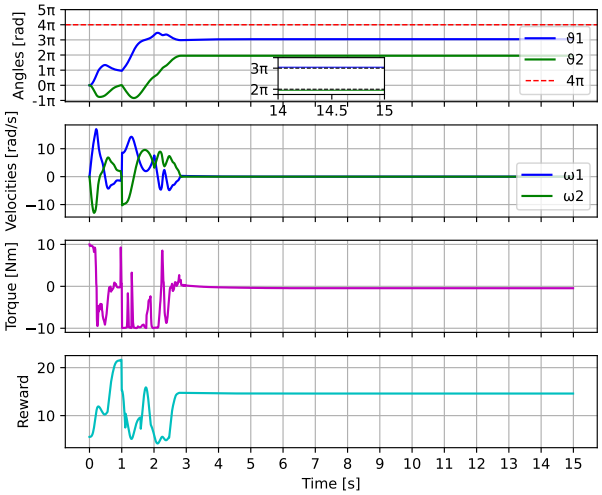

Figure 13. Best evaluation model, *Compromise* scenario.

for both $t_d = 0.5s$ and $t_d = 2s$, but violate the constraint on θ_2 in the first case. This is because the agents are trying to swing-up when the disturbance is applied and they gathered so much speed that it's not possible to decelerate the second link quickly enough. The second case proves that the agents don't forget how to correctly swing-up, although some are more precise than others. Fig. 16 shows that the *Strict* agent is the only one truly able to withstand the negative disturbance, *Compromise* falls from the upwards equilibrium after $11s$ and *Loose* makes the second link swing forever.

6. Conclusions

A training procedure that stabilizes the system after recovery while respecting the desired constraints is found and a simple degree of robustness to some disturbance parameters is shown in certain cases. Possible improvements would rely on applying more defined Constrained Reinforcement Learning approaches such as IPO (Liu et al., 2019) or CaT (Chane-Sane et al., 2024).

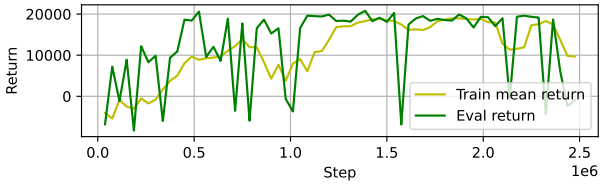

Figure 14. Learning curves, *Compromise* scenario.

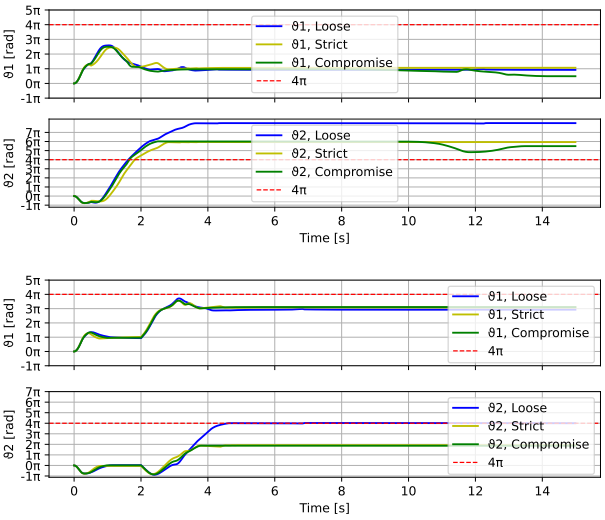

Figure 15. $\tau_{d2} > 0$, $t_d = 0.5s$ (top) and $t_d = 2s$ (bottom).

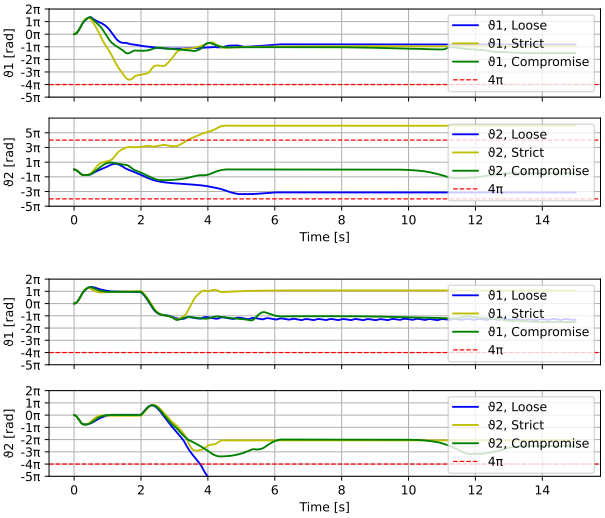

Figure 16. $\tau_{d2} < 0$, $t_d = 0.5s$ (top) and $t_d = 2s$ (bottom).

References

Calì, M., Sinigaglia, A., Turcato, N., Carli, R., and Susto, G. A. Ai olympics challenge with evolutionary soft actor critic, 2024. URL https://arxiv.org/abs/2409.01104.

Chane-Sane, E., Leziart, P.-A., Flayols, T., Stasse, O., Souères, P., and Mansard, N. Cat: Constraints as terminations for legged locomotion reinforcement learning, 2024. URL https://arxiv.org/abs/2403.18765.

Kim, S.-J., Koh, K., Lustig, M., Boyd, S., and Gorinevsky, D. An interior-point method for large-scale ℓ_1-regularized least squares. *IEEE Journal of Selected Topics in Signal Processing*, 1(4):606–617, 2007. doi: 10.1109/JSTSP.2007.910971.

Liu, Y., Ding, J., and Liu, X. Ipo: Interior-point policy optimization under constraints, 2019. URL https://arxiv.org/abs/1910.09615.

Misyats, N., Harms, M., Nissov, M., Jacquet, M., and Alexis, K. Embedded safe reactive navigation for multirotors systems using control barrier functions. In *2025 International Conference on Unmanned Aircraft Systems (ICUAS)*, pp. 697–704, 2025. doi: 10.1109/ICUAS65942.2025.11007827.

Narvekar, S., Peng, B., Leonetti, M., Sinapov, J., Taylor, M. E., and Stone, P. Curriculum learning for reinforcement learning domains: A framework and survey, 2020. URL https://arxiv.org/abs/2003.04960.